Vietnam National University of HCMC

International University

School of Computer Science and Engineering

P: Master of Information Technology

C: Advanced Computer Graphics (IT516)

Lecturer: Assoc. Prof. Nguyen Van Sinh

Email: nvsinh@hcmiu.edu.vn https://it.hcmiu.edu.vn/user/nvsinh/

OpenGL Tutorials

April 10th 2024

Acknowledgement

- This slide is adopted from a Lecture note on Programming with OpenGL by Professor Edward Angel.
- Sumanta Guha, Computer Graphics Through OpenGL From Theory to Experiments. Third edition, 2019, by Taylor & Francis Group, LLC

Objectives

- Development of the OpenGL API
- OpenGL Architecture
  - OpenGL as a state machine
- Functions
  - Types
  - Formats
- Simple programs

What is OpenGL?

- A low-level graphics rendering and imaging library
  - only includes operations which can be accelerated
- A layer of abstraction between graphics hardware and an application program
- An API to produce high-quality, color images of 3D objects

What is OpenGL?

- A procedural rather than a descriptive graphics language
- An OS and Hardware platform independent
  - X Window System under UNIX
  - Microsoft Windows or Windows NT
  - IBM OS/2
  - Apple Mac OS

OpenGL Features

- Texture mapping
- z-buffering
- Double buffering
- Lighting effects
- Smooth shading
- Material properties
- Alpha blending
- Transformation matrices

Texture Mapping

- the ability to apply an image to graphics surface
- use to rapidly generate realistic images without having to specify an excessive amount of detail
- ie. create a wooden floor by painting the floor’s rectangular surface with a wood grain texture

z-buffering

- the ability to calculate the distance from the viewer's location
- make it easy for the program to automatically remove surfaces or parts of surface that are hidden from view
- At the start, enable the Z-buffer:
  - glEnable(GL_DEPTH_TEST);
- Before each drawing, clear the Z-buffer:
  - glClear**(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);**

Double buffering

- support for smooth animation using double buffering
- drawing into the back buffer while displaying the front buffer and then swapping the buffers when you are ready to display.
- Enable double-buffering:
  - auxInitDisplayMode (AUX_DOUBLE | ....);

Double buffering

- Your drawing is done in the hidden buffer
- After drawing, swap buffers:
  - auxSwapBuffers();
- Double buffer = half the color bits.
  - 4 bits (=16 colors) on a 8 bit display…

Lighting effects

- the ability to calculate the effects on the lightness of a surface's color when different lighting models are applied to the surface from one or more light
- Three steps:
  - Enable lighting
  - Specify the lights
  - Specify the materials
- Advanced user:
  - local/infinite viewpoint
  - two sided lighting

Smooth shading

- the ability to calculate the shading effects that occur when light hits a surface at an angle and results in subtle color differences across the surface
- this effect is important for making a model look "*realistic*"

Material Properties

- the ability to specify the material properties of a surface
  - dullness
  - shininess

Alpha Blending

- the ability to specify an alpha or “opacity” value in addition to regular RGB value

Transformation Matrices

- the ability to change the location, size and perspective of an object in 3D coordinate space

How OpenGL works

- the same way that GDI (Graphics Device Interface) work
- whenever a program makes an OpenGL call, the OPENGL32 and GLU32 DLLs are loaded.

Limitation of OpenGL

Microsoft Generic implementation of OpenGL

- no direct support for printing OpenGL graphics to a monochrome printer or a color printer with less than 4 bit planes of color
- hardware palettes for various windows are not supported
- some OpenGL features are not implemented, including stereoscopic images, auxiliary buffers, and alpha bit planes.

OpenGL

- The success of GL lead to OpenGL (1992), a platform-independent API that was
  - Easy to use
  - Close enough to the hardware to get excellent performance
  - Focus on rendering
  - Omitted windowing and input to avoid window system dependencies

OpenGL Evolution

- Controlled by an Architectural Review Board (ARB)
  - Members include SGI, Microsoft, Nvidia, HP, 3DLabs,IBM,…….
  - Relatively stable (present version 1.4)
    - Evolution reflects new hardware capabilities
      - **3D texture mapping and texture objects**
      - **Vertex programs**
  - Allows for platform specific features through extensions

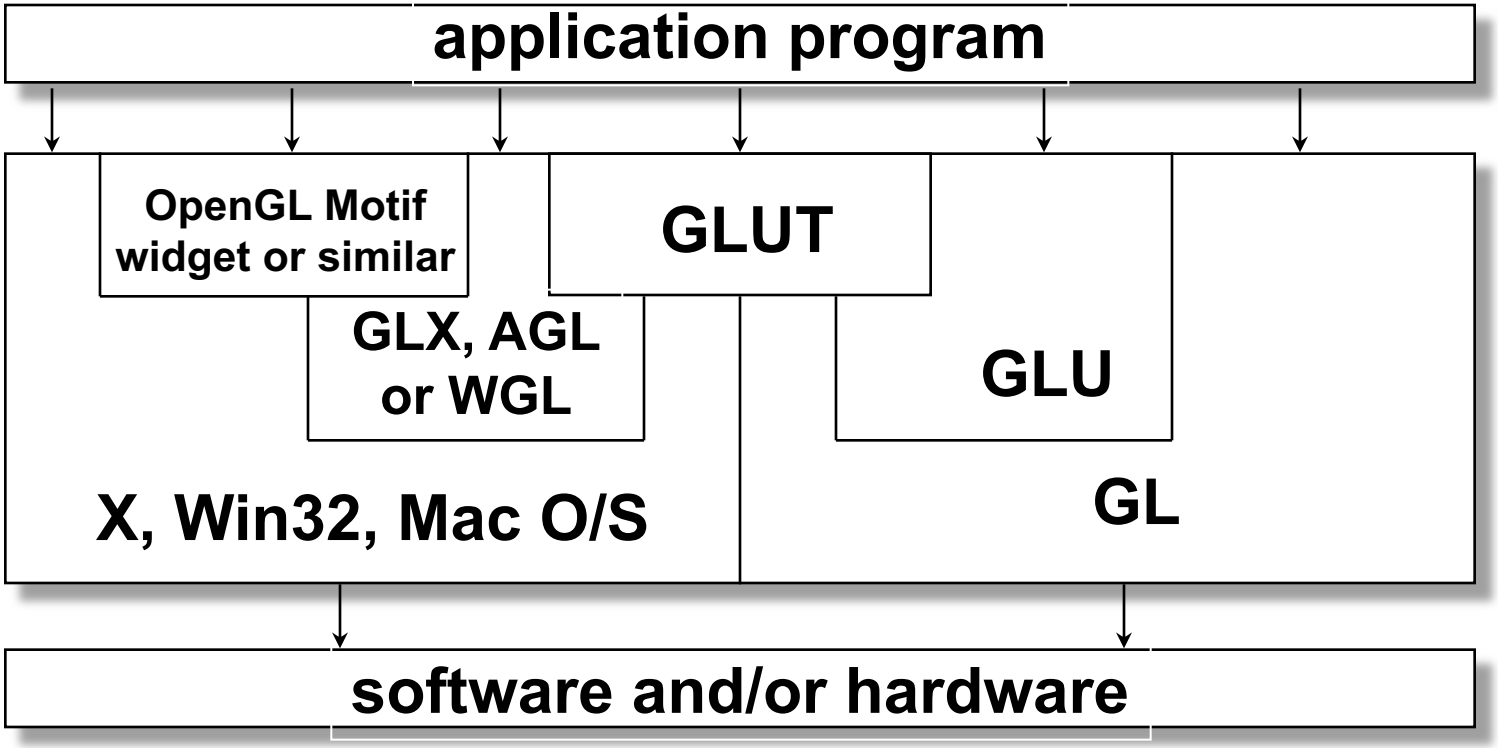
OpenGL Libraries

- OpenGL core library
  - OpenGL32 on Windows
  - GL on most unix/linux systems
- OpenGL Utility Library (GLU)
  - Provides functionality in OpenGL core but avoids having to rewrite code
- Links with window system
  - GLX for X window systems
  - WGL for Widows
  - AGL for Macintosh

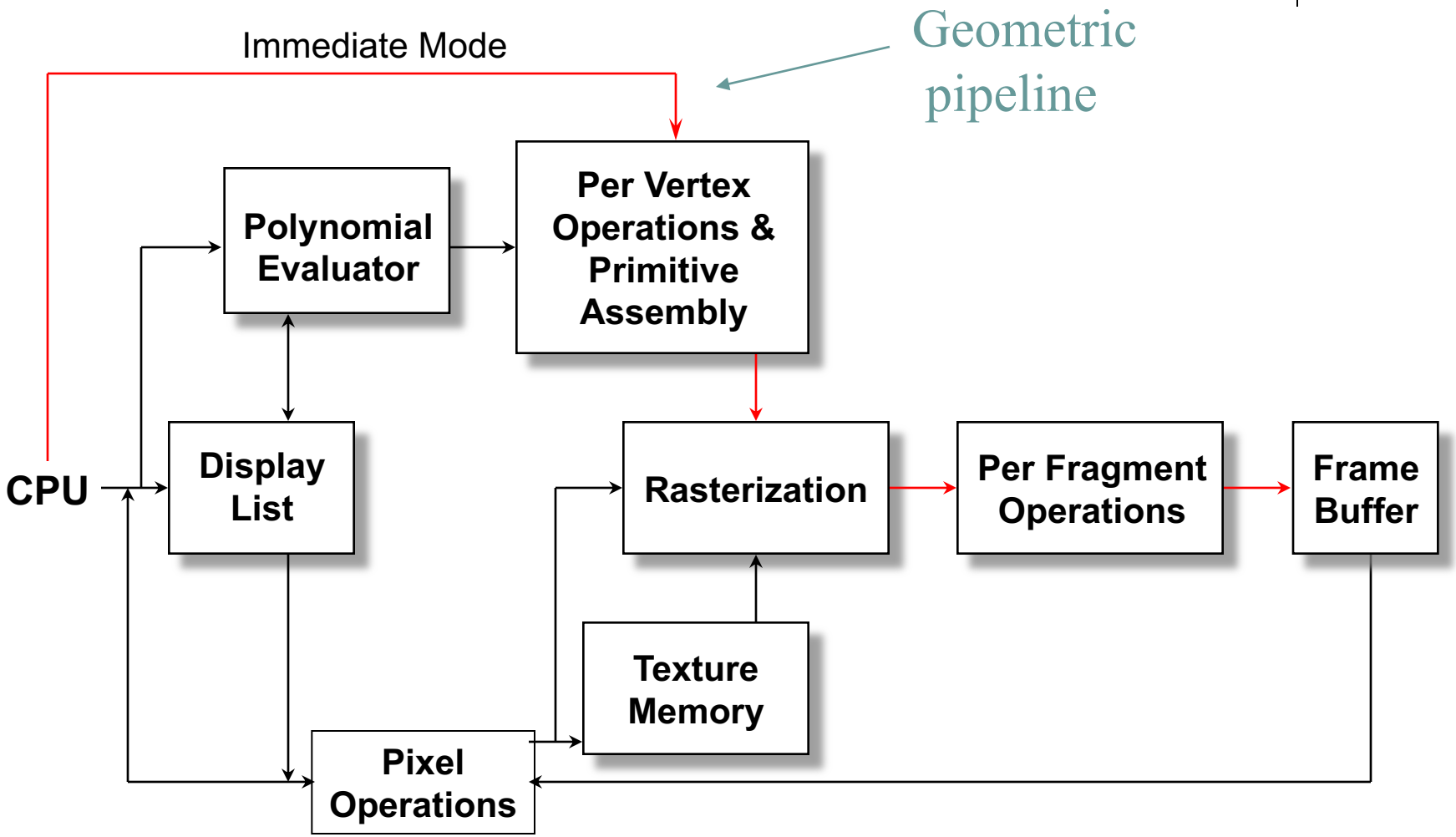
GLUT

- OpenGL Utility Library (GLUT)
  - Provides functionality common to all window systems
    - Open a window
    - Initialize OpenGL State
    - Get input from mouse and keyboard
    - Menus
    - Event-driven
  - Code is portable but GLUT lacks the functionality of a good toolkit for a specific platform
    - Slide bars
  - Not official part of OpenGL

Software Organization



OpenGL Architecture



OpenGL Functions

- Primitives
  - Points
  - Line Segments
  - Polygons
- Attributes
- Transformations
  - Viewing
  - Modeling
- Control
- Input (GLUT)

OpenGL State

- OpenGL is a state machine
- OpenGL functions are of two types
  - Primitive generating
    - Can cause output if primitive is visible
    - How vertices are processes and appearance of primitive are controlled by the state
  - State changing
    - Transformation functions
    - Attribute functions

Lack of Object Orientation

- OpenGL is not object oriented so that there are multiple functions for a given logical function, e.g. **`glVertex3f`**, **`glVertex2i`**, **`glVertex3dv`**,…..
- Underlying storage mode is the same
- Easy to create overloaded functions in C++ but issue is efficiency

OpenGL Command Notation

```
void glSomeFunction {3} {bsifd} {v} (arguments);
```

- **the first optional term in curly braces indicates that this function takes 3 arguments.**
- **the second sets of braces indicates that this function takes 5 possible argument types**
  - **b = byte, s = short, l = integer, f = float, d = double**
- **the last term in curly braces indicate that a vector form of the command also exists.**

OpenGL Command Notation

glVertex3fv(...)

Number of components

```
2 - (x,y)
3 - (x,y,z)
4 - (x,y,z,w)
```

Data Type

```
b  - byte
ub - unsigned byte
s  - short
us - unsigned short
i  - int
ui - unsigned int
f  - float
d  - double
```

Vector

omit "v" for scalar form

glVertex2f(x, y)

Preliminaries

- Header files

```
#include <GL/gl.h>
#include <GL/glu.h>
#include <GL/glut.h>
```

- GL enumerated types
  - for platform independence

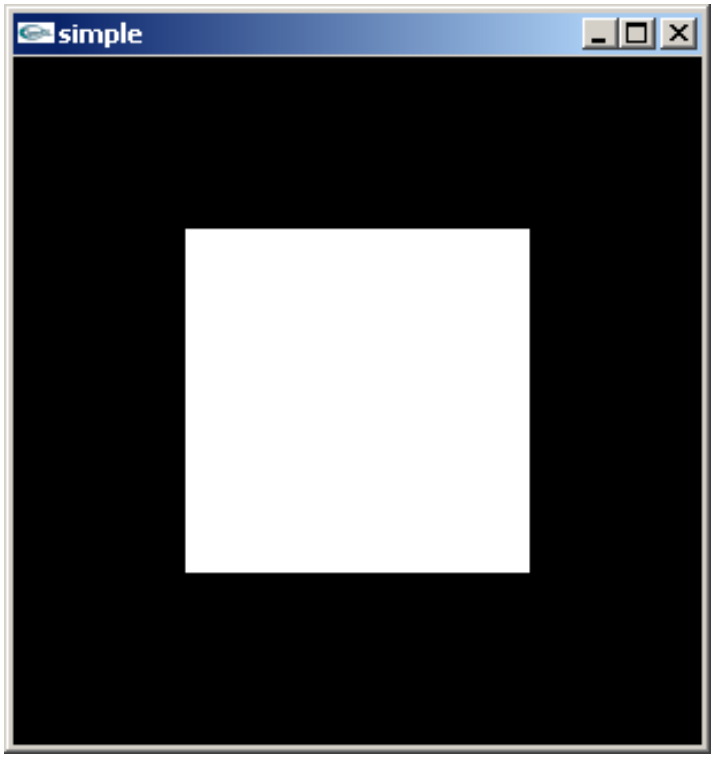
```
GLbyte, GLshort, GLushort, GLint, GLuint,
  GLsizei, GLfloat, GLdouble, GLclampf,
  GLclampd, GLubyte, GLboolean, GLenum,
  GLbitfield
```

OpenGL #defines

- Most constants are defined in the include files **`gl.h`**, **`glu.h`** and **`glut.h`**
  - Note **`#include <glut.h>`** should automatically include the others
  - Examples
    - **`glBegin(GL_PLOYGON)`**
    - **`glClear(GL_COLOR_BUFFER_BIT)`**
- include files also define OpenGL data types: **`Glfloat`**, **`Gldouble`**,….

A Simple Program

Generate a square on a solid background



simple.c

```
#include <glut.h>
void mydisplay(){
     glClear(GL_COLOR_BUFFER_BIT);
      glBegin(GL_POLYGON);
            glVertex2f(-0.5, -0.5);
            glVertex2f(-0.5, 0.5);
            glVertex2f(0.5, 0.5);
            glVertex2f(0.5, -0.5);
      glEnd();
      glFlush();
}
int main(int argc, char** argv){
      glutCreateWindow("simple");
      glutDisplayFunc(mydisplay);
      glutMainLoop();
}
```

simple.java

```
import com.jogamp.opengl.GL2;
import com.jogamp.opengl.GLAutoDrawable;
import com.jogamp.opengl.GLCapabilities;
import com.jogamp.opengl.GLEventListener;
import com.jogamp.opengl.GLProfile;
import com.jogamp.opengl.awt.GLCanvas;
import javax.swing.JFrame;
import com.jogamp.opengl.util.FPSAnimator;
public class simple implements GLEventListener {
    private float rtri;  //for angle of rotation
    @Override
    public void display(GLAutoDrawable drawable) {
        final GL2 gl = drawable.getGL().getGL2();
        gl.glClear(GL2.GL_COLOR_BUFFER_BIT | GL2.GL_DEPTH_BUFFER_BIT); // Clear The Screen And The Depth Buffer
        gl.glLoadIdentity();                // Reset The View
        gl.glRotatef(rtri, 0.0f, 1.0f, 0.0f); //square rotation
        gl.glBegin(GL2.GL_POLYGON);
        gl.glColor3f(1.0f, 0.0f,0.0f);
        gl.glVertex2d(-0.5, -0.5);
        gl.glVertex2d(-0.5, 0.5);
        gl.glColor3f(0.0f, 1.0f,0.0f);
        gl.glVertex2d(0.5, 0.5);
        gl.glVertex2d(0.5, -0.5);
        gl.glEnd();
        gl.glFlush();
        rtri += 0.2f;  //assigning the angle
    }
```

simple.java

```
public static void main(String[] args) {
    //getting the capabilities object of GL2 profile
    final GLProfile profile = GLProfile.get(GLProfile.GL2);
    GLCapabilities capabilities = new GLCapabilities(profile);
    // The canvas
    final GLCanvas glcanvas = new GLCanvas(capabilities);
    simple triangle = new simple();
    glcanvas.addGLEventListener(triangle);
    glcanvas.setSize(400, 400);
    //creating frame
    final JFrame frame = new JFrame("Rotating Triangle");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    //adding canvas to it
    frame.getContentPane().add(glcanvas);
    frame.setSize(frame.getContentPane().getPreferredSize());
    frame.setVisible(true);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    //Instantiating and Initiating Animator
    final FPSAnimator animator = new FPSAnimator(glcanvas, 300, true);
    animator.start();
  } //end of main
} //end of class
```

Event Loop

- Note that the program defines a *display callback* function named **`mydisplay`**
  - Every glut program must have a display callback
  - The display callback is executed whenever OpenGL decides the display must be refreshed, for example when the window is opened
  - The **`main`** function ends with the program entering an event loop

Defaults

- `simple.c` is too simple
- Makes heavy use of state variable default values for
  - Viewing
  - Colors
  - Window parameters
- Next version will make the defaults more explicit

Notes on compilation

- See website and ftp for examples
- Unix/linux
  - Include files usually in …/include/GL
  - Compile with –lglut –lglu –lgl loader flags
  - May have to add –L flag for X libraries
  - Mesa implementation included with most linux distributions
  - Check web for latest versions of Mesa and glut

Compilation on Windows

- Visual C++
  - Get glut.h, glut32.lib and glut32.dll from web
  - Create a console application
  - Add opengl32.lib, glut32.lib, glut32.lib to project settings (under link tab)
- Borland C similar
- Cygwin (linux under Windows)
  - Can use gcc and similar makefile to linux
  - Use –lopengl32 –lglu32 –lglut32 flags

The Main Program

- We begin with the basic elements of how to create a window. OpenGL was intentionally designed to be independent of any Specific window system. As a result, a number of the basic window operations are not provided in OpenGL.

- Therefore, a separate library called **GLUT** or *OpenGL Utility Toolkit* was created to provide these functions.

- Basically, **GLUT** provides the necessary tools for requesting windows to be created and providing interaction with I/O devices

Program Structure

- Most OpenGL programs have a similar structure that consists of the following functions
  - **`main()`**:
    - defines the callback functions
    - opens one or more windows with the required properties
    - enters event loop (last executable statement)
  - **`init()`**: sets the state variables
    - viewing
    - Attributes
  - callbacks
    - Display function
    - Input and window functions

main.c

```
#include <GL/glut.h>

int main(int argc, char** argv)
{
  glutInit(&argc,argv);
  glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
  glutInitWindowSize(500,500);
  glutInitWindowPosition(0,0);
  glutCreateWindow("simple");
  glutDisplayFunc(mydisplay);

  init();

  glutMainLoop();
}
```

includes gl.h

define window properties

display callback

set OpenGL state

enter event loop

GLUT functions

- **`glutInit`** allows application to get command line arguments and initializes system
- **`gluInitDisplayMode`** requests properties of the window (the *rendering context*)
  - RGB color
  - Single buffering
  - Properties logically ORed together
- **`glutWindowSize`** in pixels
- **`glutWindowPosition`** from top-left corner of display
- **`glutCreateWindow`** create window with title “simple”
- **`glutDisplayFunc`** display callback
- **`glutMainLoop`** enter infinite event loop

glutInit()

- The arguments allows application to get command line arguments (argc and argv) and initializes system
- This procedure must be called before any others.
- It processes (and removes) command-line arguments that may be of interest to GLUT and the window system and does general initialization of GLUT and OpenGL.

glutInitDisplayMode()

- This function performs initializations informing OpenGL how to set up its frame buffer.
- The argument to glutInitDisplayMode() is a logical-or (using the operator "|") of a number of possible options, which are given in Table 1.

Display Mode	Meaning
GLUT_RGB	Use RGB colors
GLUT_RGBA	Use RGB plus α (for transparency)
GLUT_INDEX	Use colormapped colors (not recommended)
GLUT_DOUBLE	Use double buffering (recommended)
GLUT_SINGLE	Use single buffering (not recommended)
GLUT_DEPTH	Use depth buffer (for hidden surface elim.)

Table 1: Arguments to `glutInitDisplayMode()`.

glutInitWindowSize()

- This command specifies the desired width and height of the graphics window. The general form is:

  glutInitWindowSize(int width, int height)

- The values are given in numbers of pixels.

glutInitPosition()

- This command specifies the location of the upper left corner of the graphics window. The form is

  glutInitWindowPosition(int x, int y)

- where the (*x, y*) coordinates are given relative to the upper left corner of the display. Thus, the arguments (0, 0) places the window in the upper left corner of the display.

glutCreateWindow()

- This command actually creates the graphics window. The general form of the command is

  glutCreateWindowchar(*title)

- where title is a character string. Each window has a title, and the argument is a string which specifies the window's title.

init.c

black clear color

opaque window

```
void init()
{
  glClearColor (0.0, 0.0, 0.0, 1.0);

  glColor3f(1.0, 1.0, 1.0);

  glMatrixMode (GL_PROJECTION);
  glLoadIdentity ();
  glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
}
```

fill with white

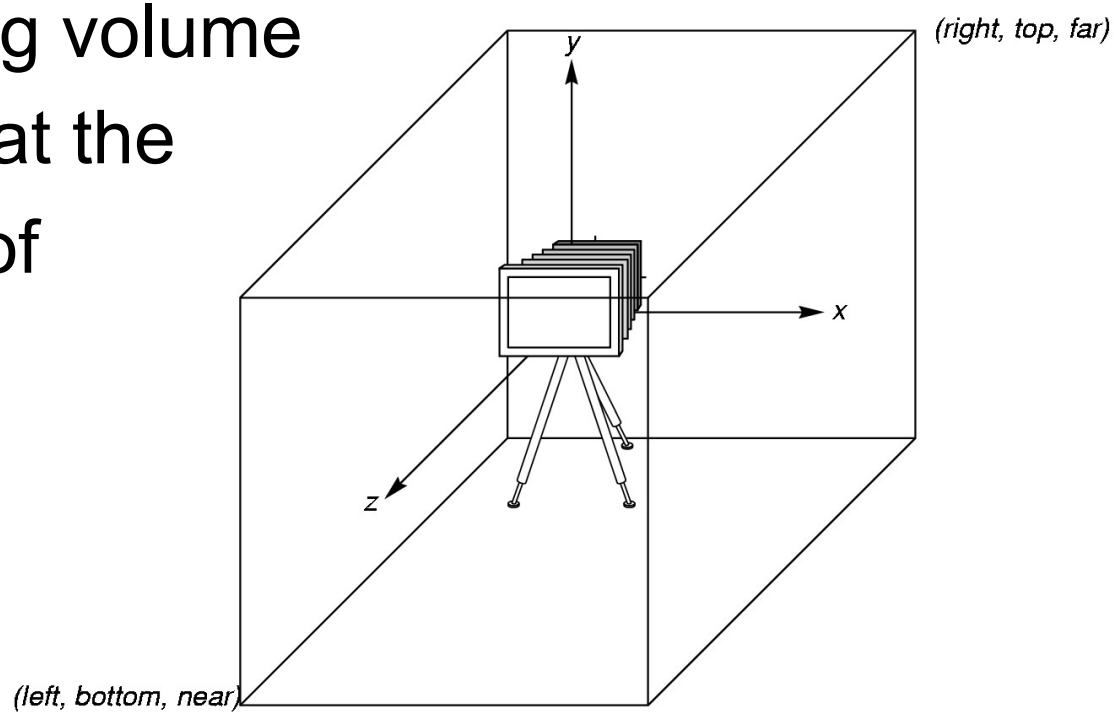
viewing volume

Coordinate Systems

- The units of in **`glVertex`** are determined by the application and are called *world* or *problem coordinates*
- The viewing specifications are also in world coordinates and it is the size of the viewing volume that determines what will appear in the image
- Internally, OpenGL will convert to *camera coordinates* and later to *screen coordinates*

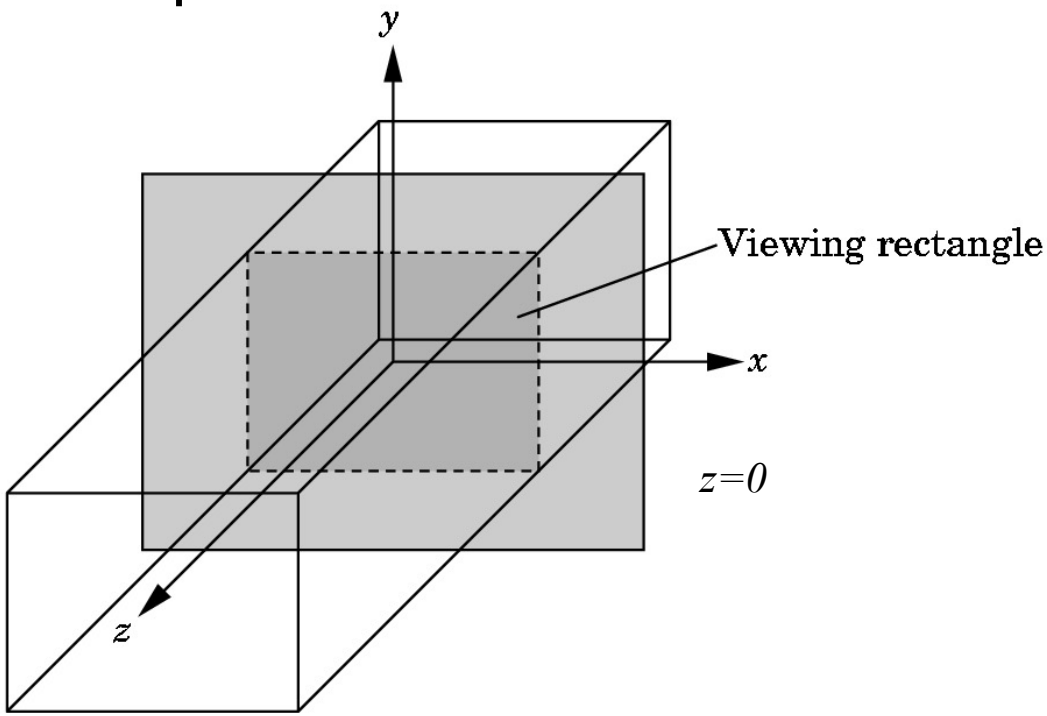
OpenGL Camera

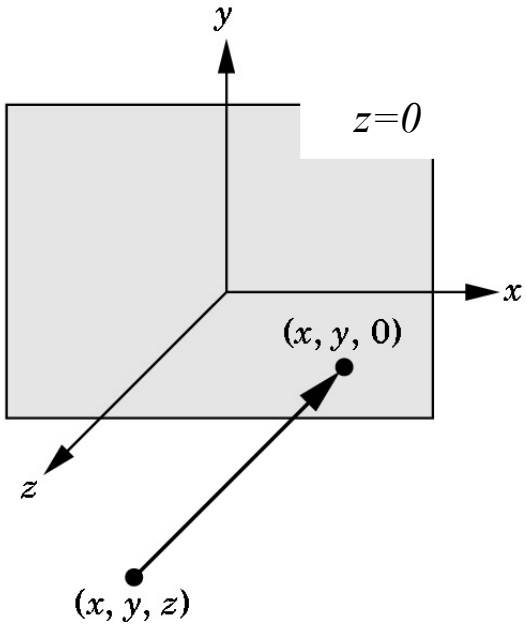
- OpenGL places a camera at the origin pointing in the negative z direction
- The default viewing volume is a box centered at the origin with a side of length 2



Orthographic Viewing

In the default orthographic view, points are projected forward along the z axis onto the plane $z=0$





Transformations and Viewing

- In OpenGL, the projection is carried out by a projection matrix (transformation)
- There is only one set of transformation functions so we must set the matrix mode first

```
glMatrixMode (GL_PROJECTION)
```

- Transformation functions are incremental so we start with an identity matrix and alter it with a projection matrix that gives the view volume

```
glLoadIdentity ();
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

Two- and three-dimensional viewing

- In **`glOrtho(left, right, bottom, top, near, far)`** the near and far distances are measured **from** the camera
- Two-dimensional vertex commands place all vertices in the plane z=0
- If the application is in two dimensions, we can use the function

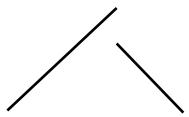
  `gluOrtho2D(left, right,bottom,top)`
- In two dimensions, the view or clipping volume becomes a *clipping window*

mydisplay.c

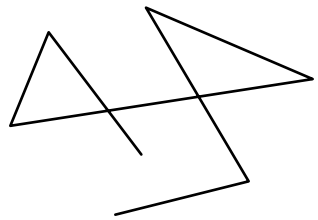
```
void mydisplay()
{
  glClear(GL_COLOR_BUFFER_BIT);
  glBegin(GL_POLYGON);
        glVertex2f(-0.5, -0.5);
        glVertex2f(-0.5, 0.5);
        glVertex2f(0.5, 0.5);
        glVertex2f(0.5, -0.5);
  glEnd();
  glFlush();
}
```

OpenGL Primitives

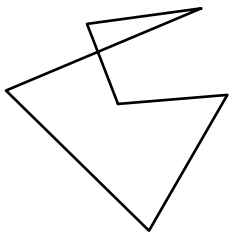
GL_POINTS



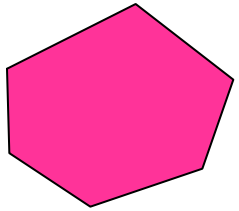
GL_LINES



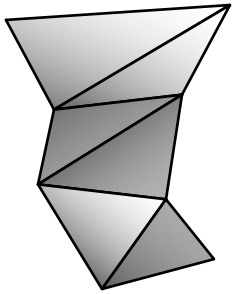
GL_LINE_STRIP



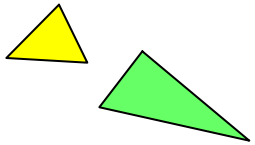
GL_LINE_LOOP



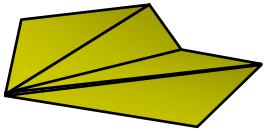
GL_POLYGON



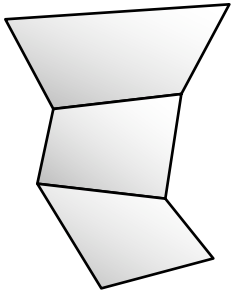
GL_TRIANGLE_STRIP



GL_TRIANGLES



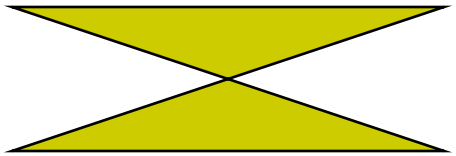
GL_TRIANGLE_FAN



GL_QUAD_STRIP

Polygon Issues

- OpenGL will only display polygons correctly that are
  - Simple: edges cannot cross
  - Convex: All points on line segment between two points in a polygon are also in the polygon
  - Flat: all vertices are in the same plane
- User program must check if above true
- Triangles satisfy all conditions



nonsimple polygon

nonconvex polygon

Attributes

- Attributes are part of the OpenGL and determine the appearance of objects
  - Color (points, lines, polygons)
  - Size and width (points, lines)
  - Stipple pattern (lines, polygons)
  - Polygon mode
    - Display as filled: solid color or stipple pattern
    - Display edges

Constructive Primitives

- The glBegin() / glEnd() **Wrappers**
- all OpenGL descriptions of primitives start with glBegin(xxx), where xxx is an OpenGL-defined constant that identifies the OpenGL primitive.

Specifying Primitives

- Primitives are described by their vertices
- *Vertex* is a point in space which is used in the construction of a geometric primitive
- Described by a *homogenous coordinate*

$$\begin{pmatrix} x & y & z & w \end{pmatrix}$$

Specifying an OpenGL Vertex

- Recall OpenGL specifies geometric primitives by its vertices

```
glVertex3f( x, y, z );
```

- Different primitives require different numbers of vertices

Drawing Points

```
glBegin(GL_POINTS);                    // selection points as
   the primitive
   glVertex3f(0.0f, 0.0f, 0.0f);       // Specify a point
   glVertex3f(50.0f, 50.0f, 50.0f);    // Specify another
   point
glEnd();                               // Done drawing
   points
```

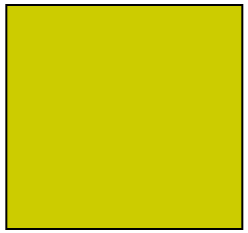
Setting the Point Size

```
void glPointSize(Glfloat size);

GLFloat sizes[2];    // Store supported point size range
GLFloat step;        // Store supported point size increments

// Get supported point size range and step size
glGetFloatv(GL_POINT_SIZE_RANGE, sizes);
glGetFloatv(GL_POINT_SIZE_GRANULARITY, &step);
```

Actually Drawing Something ...

- Here's an OpenGL sequence to draw a square centered around the origin

```
glBegin( GL_QUADS );
glVertex2f( -0.8, -0.8 );
glVertex2f(  0.8, -0.8 );
glVertex2f(  0.8,  0.8 );
glVertex2f( -0.8,  0.8 );
glEnd();
```



GL_QUADS

Adding Personality to Primitives

- *State* (or *Attributes*)
  - data required for computing colors for primitives
- Examples
  - color
  - reflectivity
  - surface texture

Specifying a Vertex's Color

- Use the OpenGL color command

```
glColor3f( r, g, b );
```

- Where you specify the color determines how the primitive is shaded
  - points only get one color

Opening a Window Using GLUT

```
void main( int argc, char** argv )
{
   glutInitWindowSize( 512, 512 );
   glutInitDisplayMode( GLUT_RGBA );
   glutCreateWindow( “my window” );

   init();

   glutDisplayFunc( drawScene );

   glutMainLoop();
}
```

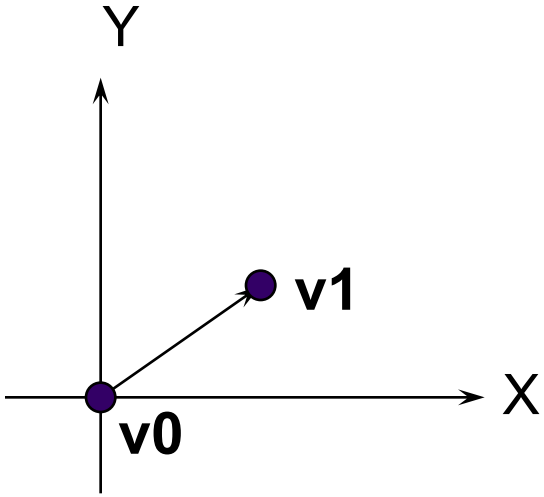
OpenGL Initalization

- We'll use the init() routine for our one-time OpenGL state initialization
  - call after window has been created, but before first rendering call

```
void init( void )
{
  glClearColor( 1.0, 0.0, 0.0, 1.0 );
}
```

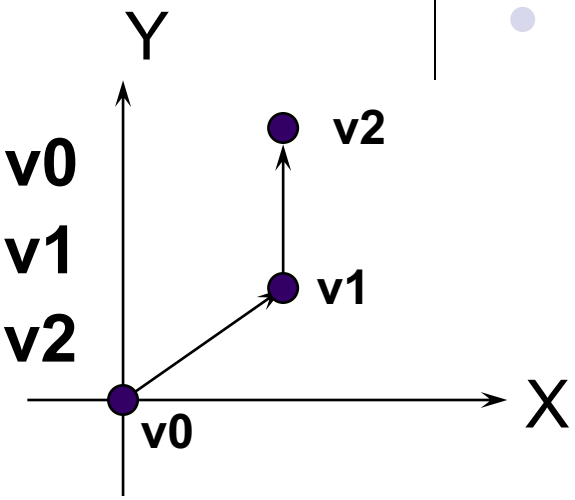
Drawing Lines in 3D

```
glBegin(GL_LINE_STRIP)
   glVertex3f(0.0f, 0.0f, 0.0f);        // v0
   glVertex3f(50.0f, 50.0f, 50.0f);     // v1
glEnd();
```

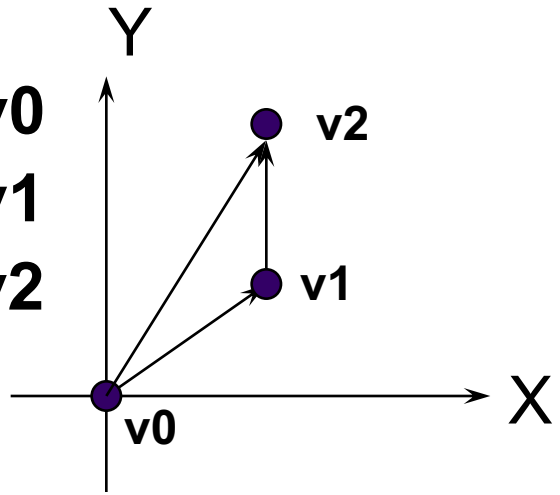


Draw Line in 3D

```
glBegin(GL_LINE_STRIP)
   glVertex3f(0.0f, 0.0f, 0.0f);        // v0
   glVertex3f(50.0f, 50.0f, 0.0f);      // v1
   glVertex3f(50.0f, 100.0f, 0.0f);     // v2
glEnd();
```




```
glBegin(GL_LINE_LOOP)
   glVertex3f(0.0f, 0.0f, 0.0f);        // v0
   glVertex3f(50.0f, 50.0f, 0.0f);      // v1
   glVertex3f(50.0f, 100.0f, 0.0f);     // v2
glEnd();
```

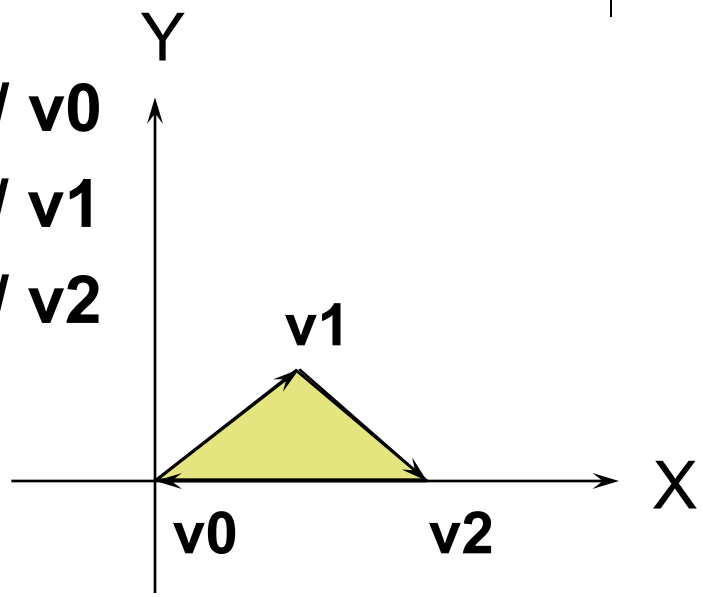


Setting the Line Width

```
void glLineWidth( GLFloat width);
GLFloat sizes[2];    // Store supported line width range
GLFloat step;        // Store supported line width increments

// Get supported line width range and step size
glGetFloatv(GL_LINE_WIDTH_RANGE, sizes);
glGetFLoatv(GL_LINE_WIDTH_GRANULARITY, &step);
```

Drawing Triangle in 3D

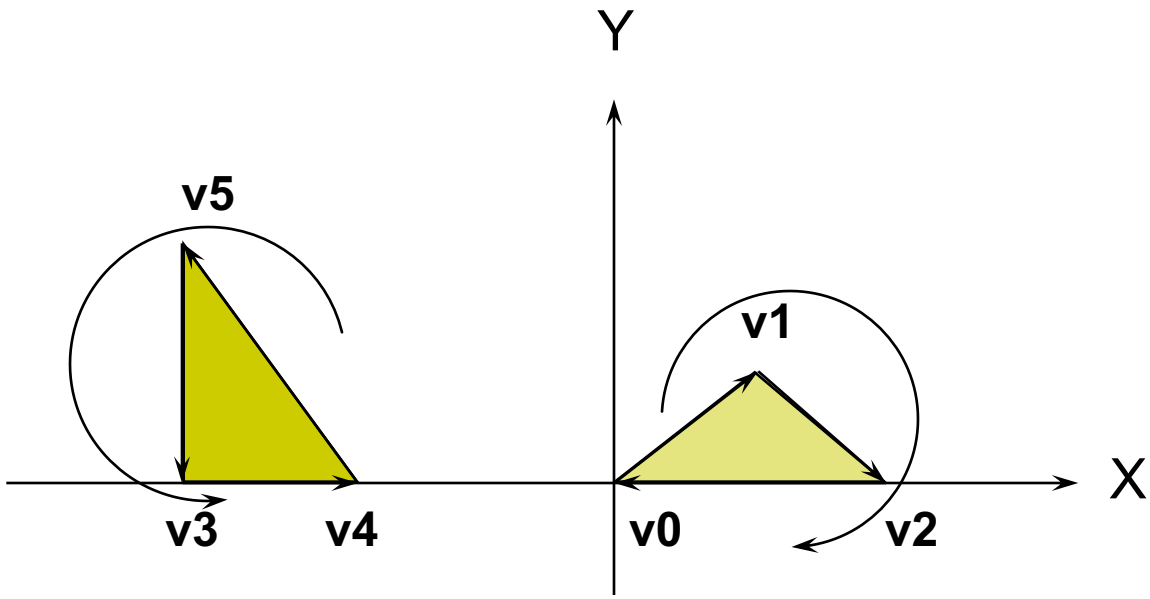
```
glBegin(GL_TRIANGLES)
   glVertex2f(0.0f, 0.0f);        // v0
   glVertex2f(25.0f, 25.0f);      // v1
   glVertex2f(50.0f, 0.0f);       // v2
glEnd();
```



Choose the Fastest Primitives for Performance Tip
Most 3D accelerated hardware is highly optimized for the drawing of triangles.

Winding

- The combination of order and direction in which the vertices are specified



Counterclockwise winding (Front Facing)

Clockwise winding (Back facing)

Winding (cont)

- OpenGL by default considers polygons that have counterclockwise winding to be front facing

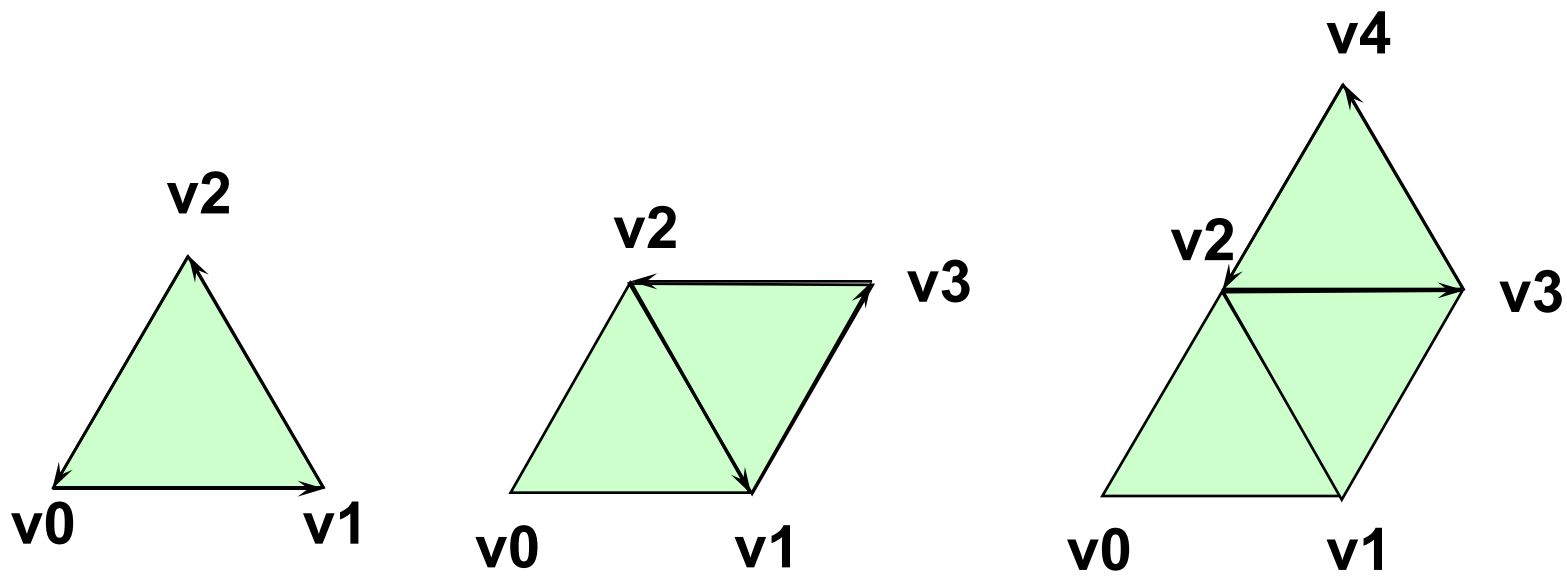
Why so important?

- you can hide the back of a polygon altogether, or give it a different color and reflective property as well

```
GLFrontFace(GL_CW);     // change default winding to clockwise
GLFrontFace(GL_CCW);    // change back to counterclockwise
```

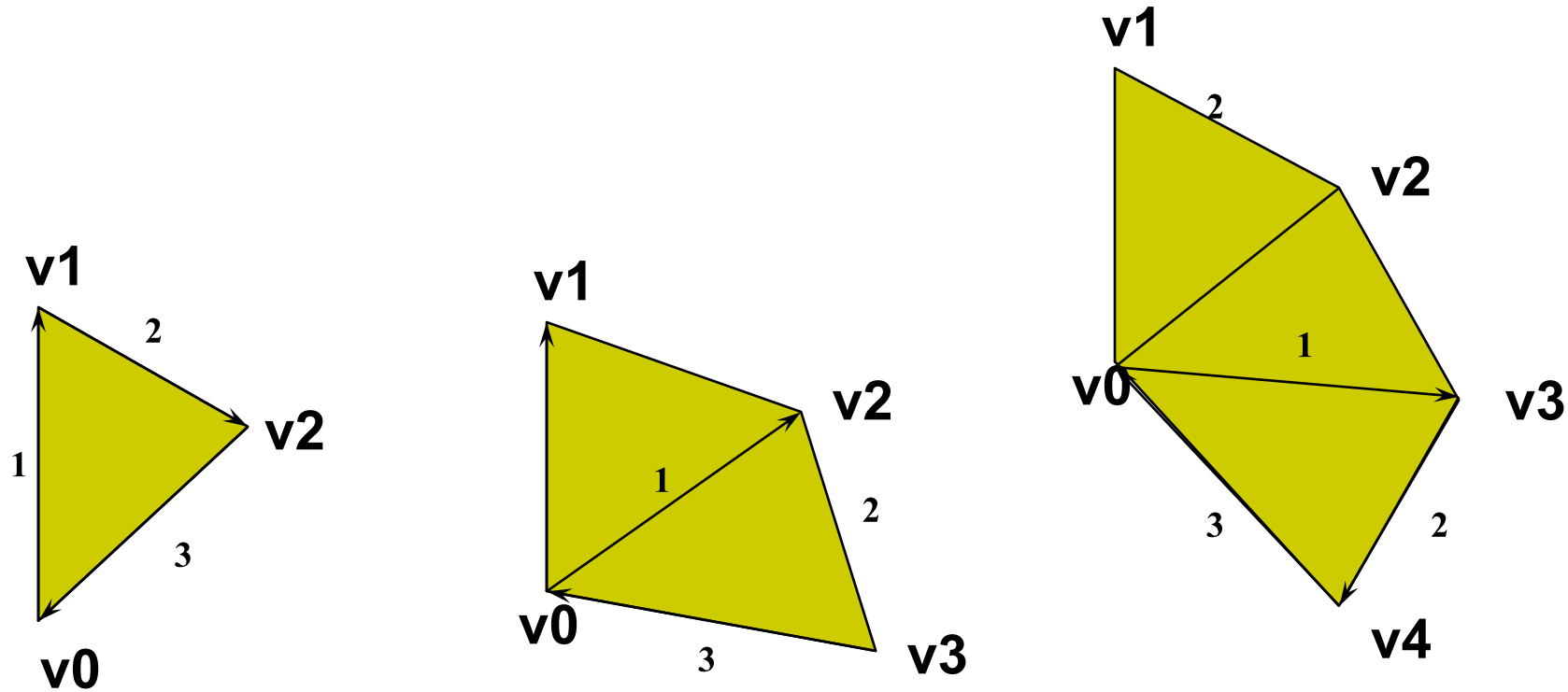
Triangle Strips

- GL_TRIANGLE_STRIP



Triangle Fans

GL_TRIANLGLE_FAN

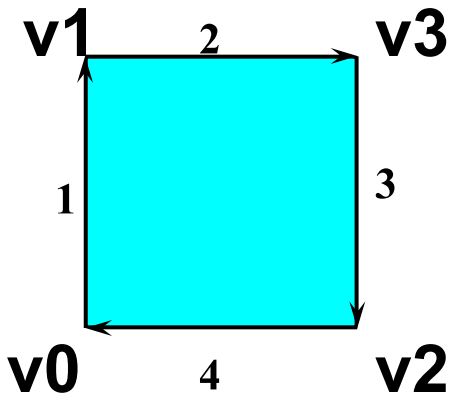


Setting Polygon Colors

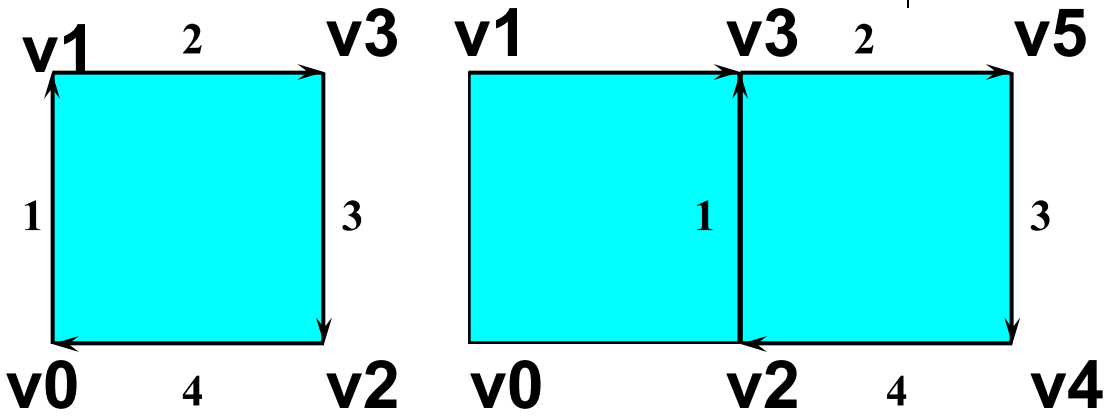
- Colors are specified per vertex, not per polygon
- glShadeModel(GL_FLAT)
- glShadeModel(GL_SMOOTH)

- GL_FLAT tells the OpenGL to fill the polygon with the solid color that was current when the polygon's last vertex was specified.
- GL_SMOOTH tells the OpenGl to shade the triangle smoothly from each vertex, attempting to interpolate the colors between those specified for each vertex

Four-Sided Polygons: Quads



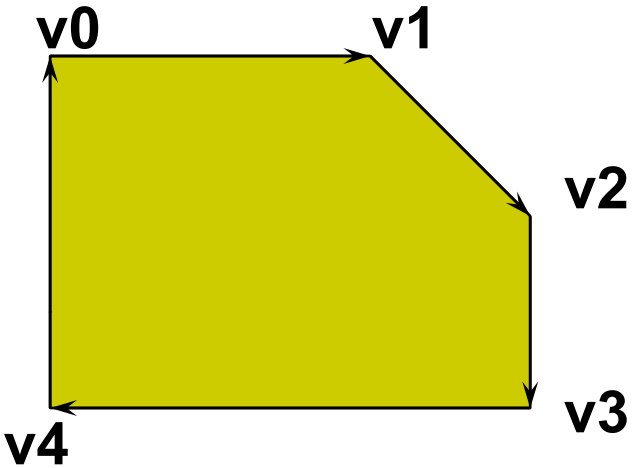
Example of GL_QUAD



Progression of GL_QUAD_STRIP

General Polygon

GL_POLYGON



Simple lighting

Enable lighting:

- glEnable(GL_LIGHTING);
- glEnable(GL_LIGHT0);

Specify light sources parameters:

- glLightfv(GL_LIGHT0,GL_AMBIENT, light_ambient);
- glLightfv(GL_LIGHT0,GL_POSITION, light_pos);
- glLightfv(GL_LIGHT0,GL_DIFFUSE, light_dif);

Plus global ambient light:

- glLightModelfv(GL_LIGHT_MODEL_AMBIENT, l_ambient);

Specifying materials

One function call for each property:

- glMaterialfv(GL_FRONT_AND_BACK,GL_AMBIENT,mat_amb);
- glMaterialfv(GL_FRONT_AND_BACK,GL_DIFFUSE,mat_diff);
- glMaterialfv(GL_FRONT_AND_BACK,GL_SPECULAR,matspec);
- glMaterialf(GL_FRONT_AND_BACK,GL_SHININESS,100.0);
- glMaterialfv(GL_FRONT_AND_BACK,GL_EMISSION,mat_emi);

Specifying materials

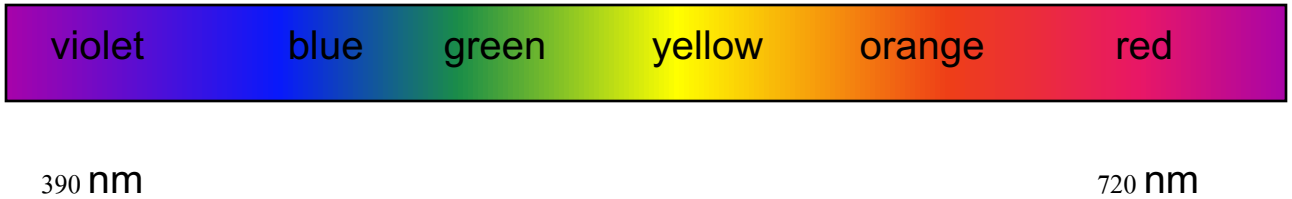
Also possible using glColor();

- glColorMaterial(GL_FRONT,GL_DIFFUSE);
- glEnable(GL_COLOR_MATERIAL);
- glColor3f(0.14,0.33,0.76);

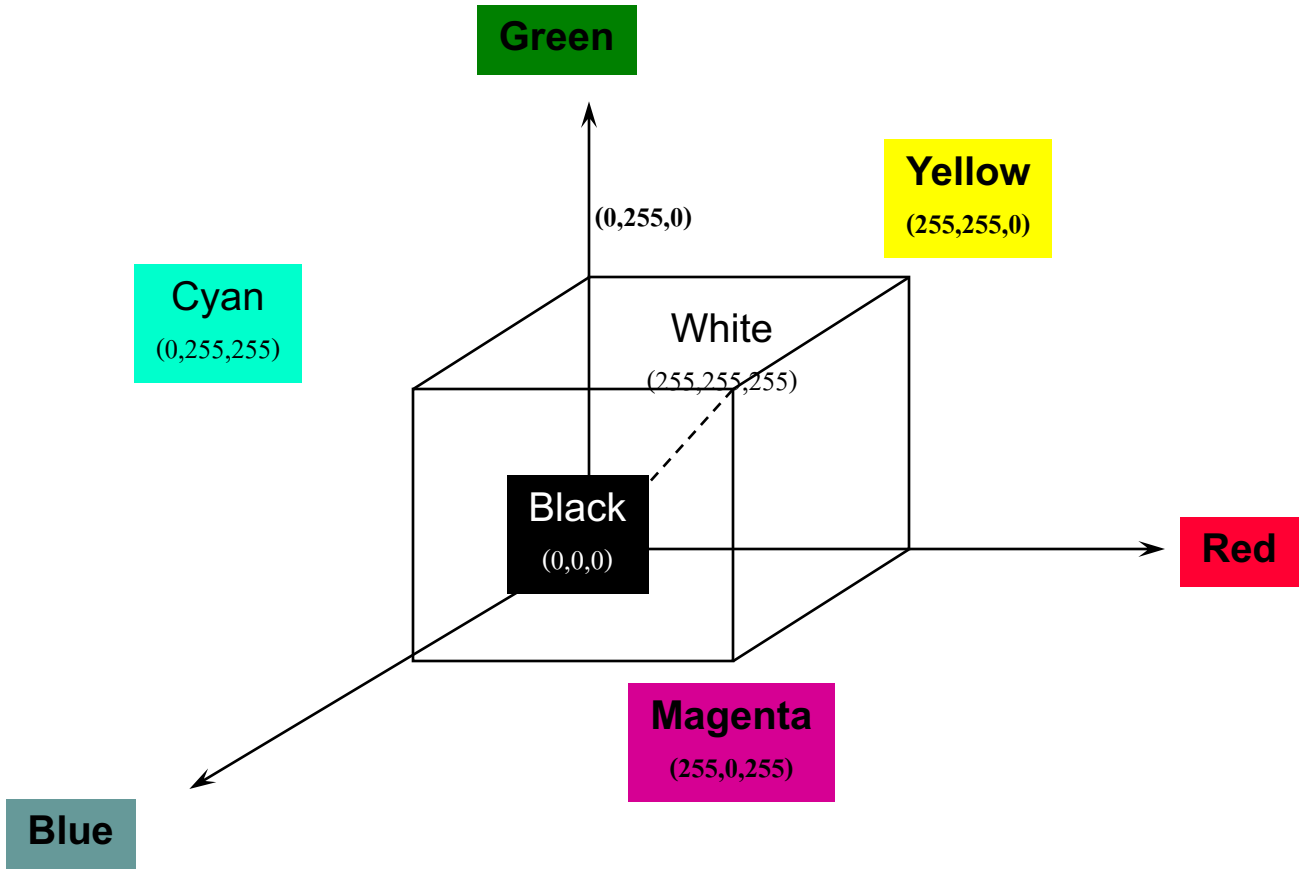
Color

OpenGL supports two colors model

- RGBA mode
- color-index mode



The Color Cube

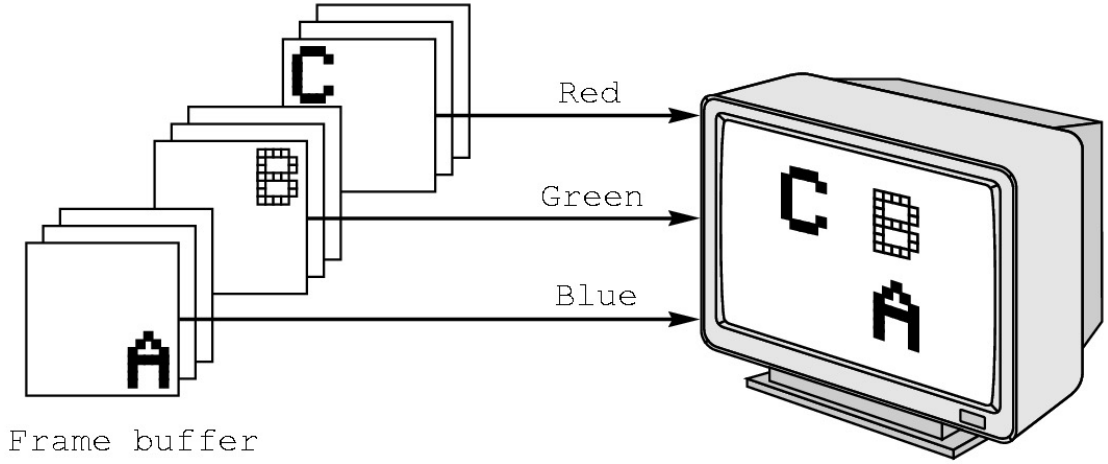


Color and Shading

- Color in RGBA mode is set by specifying the red, green, blue and alpha intensities.
- alpha = 1 //opaque
- alpha = 0 // transparent

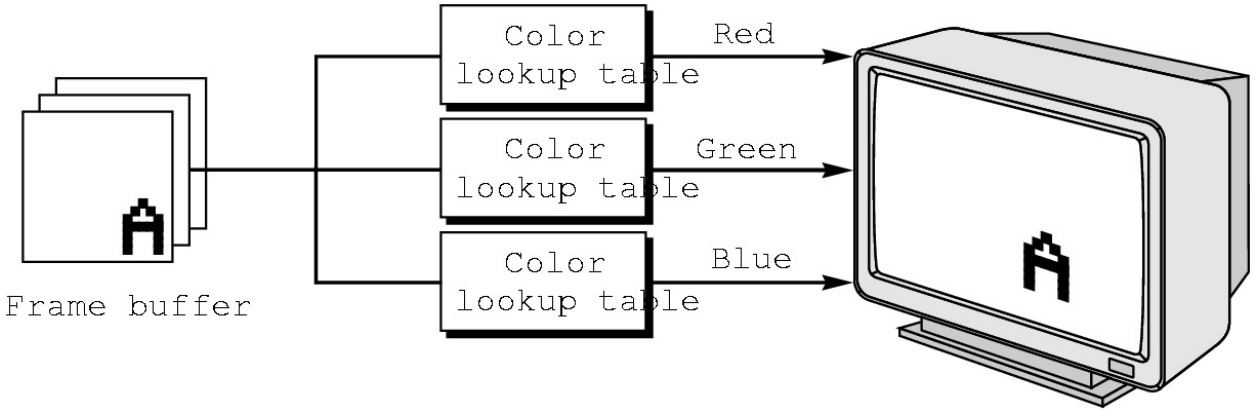
RGB color

- Each color component stored separately in the frame buffer
- Usually 8 bits per component in buffer
- Note in `glColor3f` the color values range from 0.0 (none) to 1.0 (all), while in `glColor3ub` the values range from 0 to 255



Indexed Color

- Colors are indices into tables of RGB values
- Requires less memory
  - indices usually 8 bits
  - not as important now
    - Memory inexpensive
    - Need more colors for shading



Color and State

- The color as set by **`glColor`** becomes part of the state and will be used until changed
  - Colors and other attributes are not part of the object but are assigned when the object is rendered
- We can create conceptual *vertex colors* by code such as

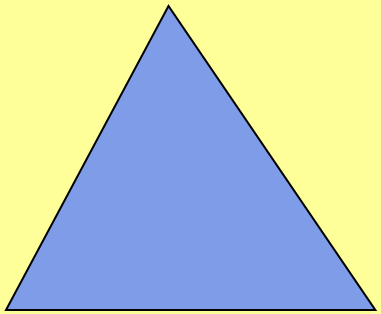
```
glColor
glVertex
glColor
glVertex
```

Setting of color attribute

- glClearColor(1.0, 1.0, 1.0, 0.0); //Clear color
- glColor3f(1.0f, 0.0f, 0.0f); // red

Example of setting color

```
glColor3f(1.0f, 0.0f, 0.0f);  // no alpha value form
glBegin( GL_TRIANGLEs);
   glVertex3f( -1.0f, 0.0f, 0.0f);
   glVertex3f(1.0f, 0.0f, 0.0f);
   glVertex3f(0.0f, 1.0f, 0.0f);
glEnd();
```



- Note that the glColor*() function can be placed inside a glBegin()/glEnd() pair. Therefore you can specify individual colors for each individual vertex

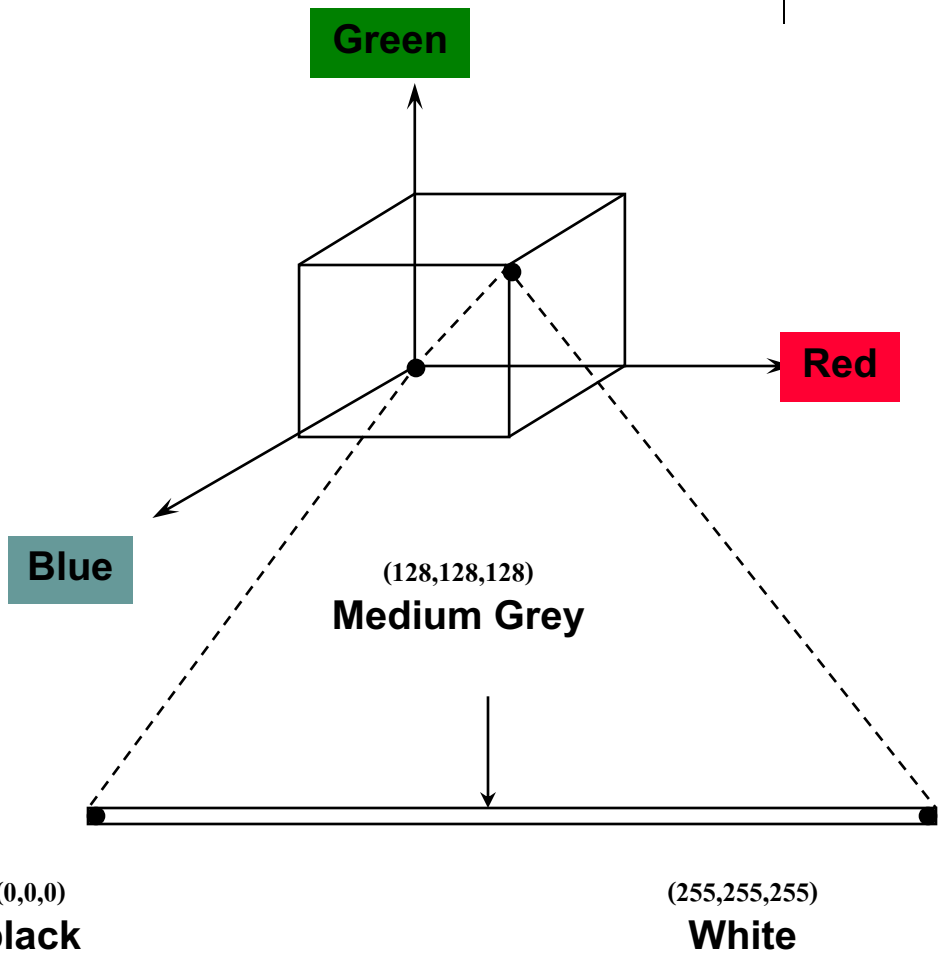
2 vertices of different color

What happen if 2 vertices have different colors?

```
glBegin( GL_TRIANGLEs);
   glColor3f(1.0f, 0.0f, 0.0f);  // red
   glVertex3f( -1.0f, 0.0f, 0.0f);
   glColor3f(0.0f, 1.0f, 0.0f);  // green
   glVertex3f(1.0f, 0.0f, 0.0f);
   glColor3f(0.0f, 0.0f, 1.0f);  // blue
   glVertex3f(0.0f, 1.0f, 0.0f);
glEnd();
```

Shading

- What color is the interior if we specify a different color for each vertex of a primitive?
- Smooth shading causes the color vary as they do through the color cube from one point to the other



Shading Model

- glShadeModel(GL_SMOOTH);
- glShadeModel(GL_FLAT);

- *Flat shading* means that no shading calculations are performed on the interior of primitives.
  - Generally the color specify by the last vertex except the GL_POLYGON primitive, the color specify by the first vertex.

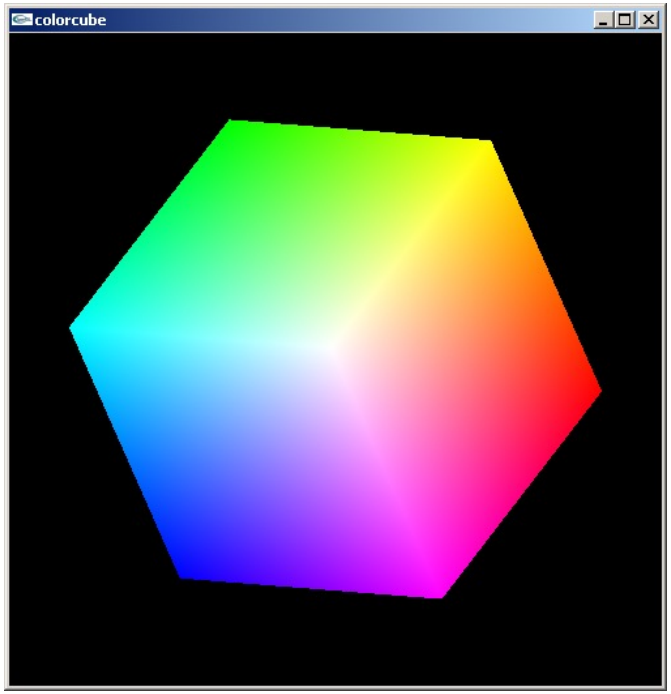
Smooth Color

- Default is *smooth* shading
  - OpenGL interpolates vertex colors across visible polygons
- Alternative is *flat shading*
  - Color of first vertex

  determines fill color
- **`glShadeModel`**

  `(GL_SMOOTH)`

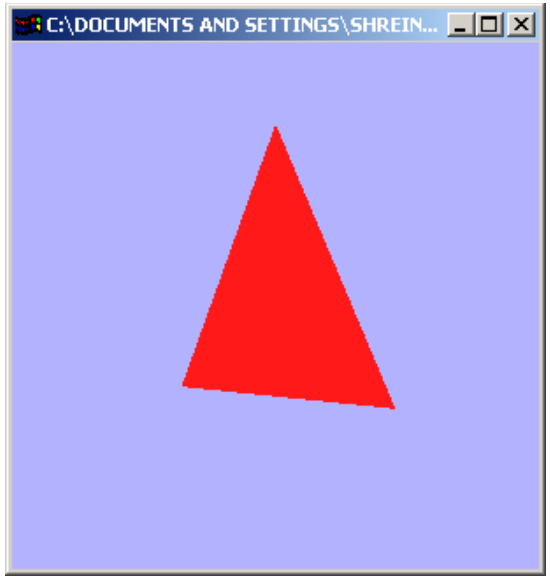
  or **`GL_FLAT`**



Flat Shading in OpenGL

- If you issue only one `glColor()` command per primitive

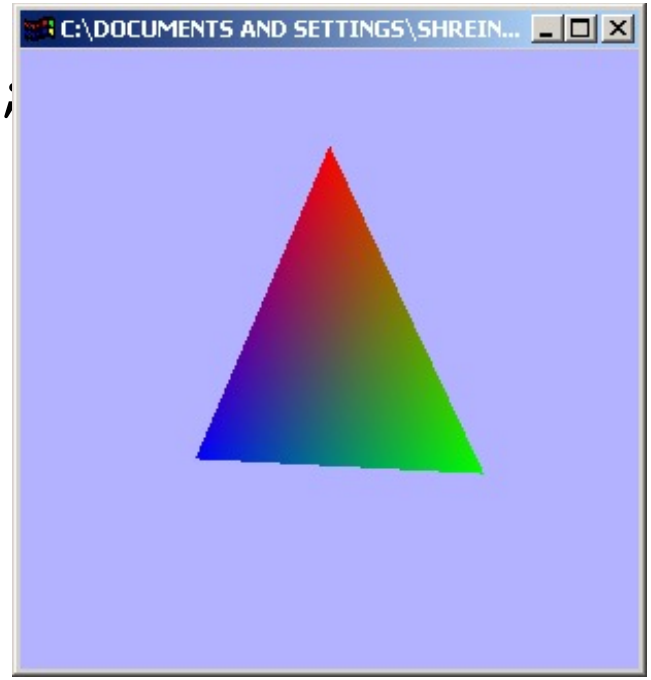
```
glColor3f( r, g, b );
glBegin( GL_TRIANGLES );
  glVertex3fv( v1 );
  glVertex3fv( v2 );
  glVertex3fv( v3 );
glEnd();
```



Gouraud Shading in OpenGL

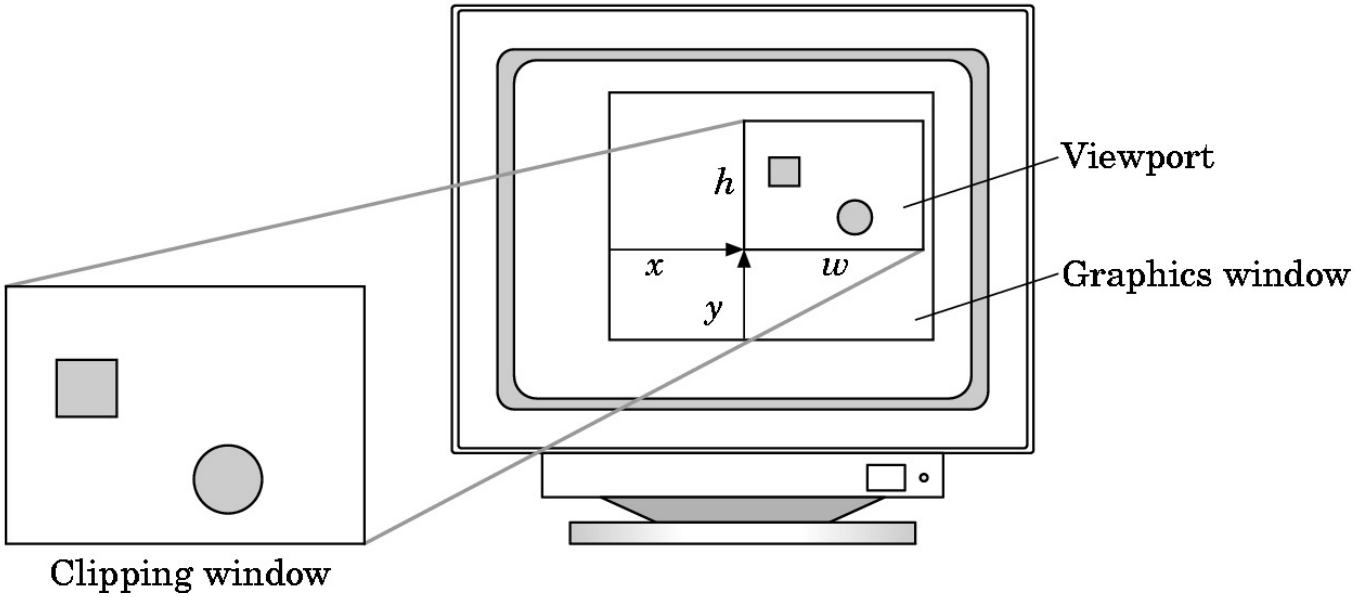
- However, to get Gouraud, issue a color per vertex

```
glBegin( GL_TRIANGLES );
  glColor3fv( c1 );
  glVertex3fv( v1 );
  glColor3fv( c2 );
  glVertex3fv( v2 );
  glColor3fv( c3 );
  glVertex3fv( v3 );
glEnd();
```



Viewports

- Do not have use the entire window for the image: **`glViewport(x,y,w,h)`**
- Values in pixels (screen coordinates)



Event-Driven Programming & Callbacks

- Virtually all interactive graphics programs are *event driven*.
- Therefore, a graphics program must be prepared at any time for input from any number of sources, including the mouse, or keyboard, or other graphics devises such as trackballs and joysticks.
- In OpenGL, this is done through the use of *callbacks.*

Callbacks

- The *callbacks* are used when the graphics program instructs the system to invoke a particular procedure or functions whenever an event of interest occurs, say, the mouse button is clicked.
- The graphics program indicates its interest, or *registers*, for various events. This involves telling the window system which event type we are interested in, and passing it the name of a procedure we have written to handle the event.

Types of callback

- Callbacks are used for two purposes:
  1. User input events
  2. System events

User input events: include things such as mouse clicks,the motion of the mouse (without clicking) that is also called *passive motion* and keyboard hits.

NB: The program is only signaled about events that happen to its window. For example, entering text into another window's dialogue box will not generate a keyboard event for the program.

Types of callback

System event: There are a number of different events that are generated by the system such as *display event, reshape event, idle event* and *timer event*.

display event **:** a special event that every OpenGL program must handle. A display event is invoked when the system senses that the contents of the window need to be redisplayed, either because: the graphics window has completed its initial creation an obscuring window has moved away, thus revealing all or part of the graphics window

Types of callback

- The program explicitly requests redrawing, by calling ***glutPostRedisplay()*** procedure.

Types of callback

- *Reshape event*: Happens whenever the window shape is altered.The callback provides information on the new size of the window.
- *Idle event:* Happens every time the system has nothing to do
- *Timer event*: Happens when after the waiting period is over

Callbacks

Table: Common callbacks and the associated registration functions

Input Event	Callback request	User callback function prototype (return void)
Mouse button	`glutMouseFunc`	`myMouse(int b, int s, int x, int y)`
Mouse motion	`glutPassiveMotionFunc`	`myMotion(int x, int y)`
Keyboard key	`glutKeyboardFunc`	`myKeyboard(unsigned char c, int x, int y)`
System Event	**Callback request**	**User callback function prototype (return void)**
(Re)display	`glutDisplayFunc`	`myDisplay()`
(Re)size window	`glutReshapeFunc`	`myReshape(int w, int h)`
Timer event	`glutTimerFunc`	`myTimer(int id)`
Idle event	`glutIdleFunc`	`myIdle()`

Callback Setup

```
int main(int argc, char** argv)
{
...
glutDisplayFunc(myDraw); // set up the callbacks
glutReshapeFunc(myReshape);
glutMouseFunc(myMouse);
glutKeyboardFunc(myKeyboard);
glutTimerFunc(20, myTimeOut, 0);
...
}
```

Callback Functions

- Callback functions depend on the function definition that we create.

Examples of Callback Functions for SE.

```
void myDraw() {                                  // called to display window
// ...insert your drawing code here ...
}
void myReshape(int w, int h) {                   // called if reshaped
windowWidth = w;                                 // save new window size
windowHeight = h;
// ...may need to update the projection ...
glutPostRedisplay();                             // request window redisplay
}
void myTimeOut(int id) {                         // called if timer event
        // ...advance the state of animation incrementally...
glutPostRedisplay();                                     // request redisplay
glutTimerFunc(20, myTimeOut, 0);                 // request next timer event
}
```

Callback Function for SE

- From the example, both the timer and reshape callback invoke the function **glutPostRedisplay().**

- This function informs OpenGL that the state of the scene has changed and should be redrawn

Example of Callback Functions for User Input Events

```
void myMouse(int b, int s, int x, int y) {
switch (b) {                                  // b indicates the button
   case GLUT_LEFT_BUTTON:
         if (s == GLUT_DOWN)      // button pressed
               // ...
         else if (s == GLUT_UP)    // button released
               // ...
         break;
   // ...                                     // other button events
   }
}
```

GLUT Parameters

GLUT Parameter Name	Meaning
GLUT_LEFT_BUTTON	left mouse button
GLUT_MIDDLE_BUTTON	middle mouse button
GLUT_RIGHT_BUTTON	right mouse button
GLUT_DOWN	mouse button pressed down
GLUT_UP	mouse button released

GLUT parameter names associated with mouse events

Example of Callback Functions for User Input Events

```
// called if keyboard key hit
void myKeyboard(unsigned char c, int x, int y) {
   switch (c) {                    // c is the key that is hit
        case ’q’:                   // ’q’ means quit
                exit(0);
        break;
        // ...              // other keyboard events
   }
}
```

Homework

1) Go to this site, Research and implement all examples

https://www3.ntu.edu.sg/home/ehchua/programming/opengl/JOGL2.0.html

2) Explore and implement yourself many exercises in the book "Computer Graphics Through OpenGL From Theory to Experiments"

3) Refer this site:
https://www.tutorialspoint.com/jogl/jogl_drawing_basics.htm